

```

/// BLE Services Documentation: Battery Level and Heart Rate Management
/// Translated from Java to Dart

import 'package:flutter_blue/flutter_blue.dart';

class BleManager {
  late BluetoothDevice _connectedDevice;
  BluetoothCharacteristic? _batteryLevelCharacteristic;
  BluetoothCharacteristic? _heartRateMeasurementCharacteristic;

  /// Connect to a Bluetooth device
  Future<void> connectToDevice(BluetoothDevice device) async {
    _connectedDevice = device;
    await device.connect(autoConnect: false);
    print('Connected to device: ${device.name}');

    // Discover services
    List<BluetoothService> services = await device.discoverServices();
    for (var service in services) {
      if (service.uuid.toString().toUpperCase() ==
'0000180F-0000-1000-8000-00805F9B34FB') {
        _batteryLevelCharacteristic = service.characteristics.firstWhere(
          (c) => c.uuid.toString().toUpperCase() ==
'00002A19-0000-1000-8000-00805F9B34FB',
          orElse: () => throw Exception('Battery Level Characteristic not
found'),
        );
      }

      if (service.uuid.toString().toUpperCase() ==
'0000180D-0000-1000-8000-00805F9B34FB') {
        _heartRateMeasurementCharacteristic =
service.characteristics.firstWhere(
          (c) => c.uuid.toString().toUpperCase() ==
'00002A37-0000-1000-8000-00805F9B34FB',
          orElse: () => throw Exception('Heart Rate Measurement Characteristic
not found'),
        );
      }
    }
  }

  /// Read Battery Level
  Future<int> readBatteryLevel() async {
    if (_batteryLevelCharacteristic == null) {
      throw Exception('Battery Level Characteristic not initialized');
    }
    var value = await _batteryLevelCharacteristic!.read();
    return value.first;
  }

  /// Subscribe to Battery Level Notifications
  Future<void> subscribeToBatteryNotifications(Function(int) onData) async {
    if (_batteryLevelCharacteristic == null) {

```

```

        throw Exception('Battery Level Characteristic not initialized');
    }
    await _batteryLevelCharacteristic!.setNotifyValue(true);
    _batteryLevelCharacteristic!.value.listen((data) {
        onData(data.first);
    });
}

/// Unsubscribe from Battery Level Notifications
Future<void> unsubscribeFromBatteryNotifications() async {
    if (_batteryLevelCharacteristic == null) {
        throw Exception('Battery Level Characteristic not initialized');
    }
    await _batteryLevelCharacteristic!.setNotifyValue(false);
}

/// Subscribe to Heart Rate Notifications
Future<void> subscribeToHeartRateNotifications(Function(int) onHeartRate)
async {
    if (_heartRateMeasurementCharacteristic == null) {
        throw Exception('Heart Rate Measurement Characteristic not initialized');
    }
    await _heartRateMeasurementCharacteristic!.setNotifyValue(true);
    _heartRateMeasurementCharacteristic!.value.listen((data) {
        int heartRate = data[1];
        onHeartRate(heartRate);
    });
}

/// Disconnect from the device
Future<void> disconnectDevice() async {
    await _connectedDevice.disconnect();
    print('Disconnected from device: ${_connectedDevice.name}');
}

void main() async {
    final BleManager bleManager = BleManager();
    final FlutterBlue flutterBlue = FlutterBlue.instance;

    // Scan for devices
    flutterBlue.scan(timeout: Duration(seconds: 5)).listen((scanResult) async {
        if (scanResult.device.name == 'YourDeviceName') {
            await flutterBlue.stopScan();

            // Connect to the device
            await bleManager.connectToDevice(scanResult.device);

            // Read battery level
            int batteryLevel = await bleManager.readBatteryLevel();
            print('Battery Level: $batteryLevel%');

            // Subscribe to heart rate notifications
            await bleManager.subscribeToHeartRateNotifications((heartRate) {

```

```
        print('Heart Rate: $heartRate bpm');
    });
}
}
```